STAT 545 HW 7

Bowen Zheng

2026-03-04

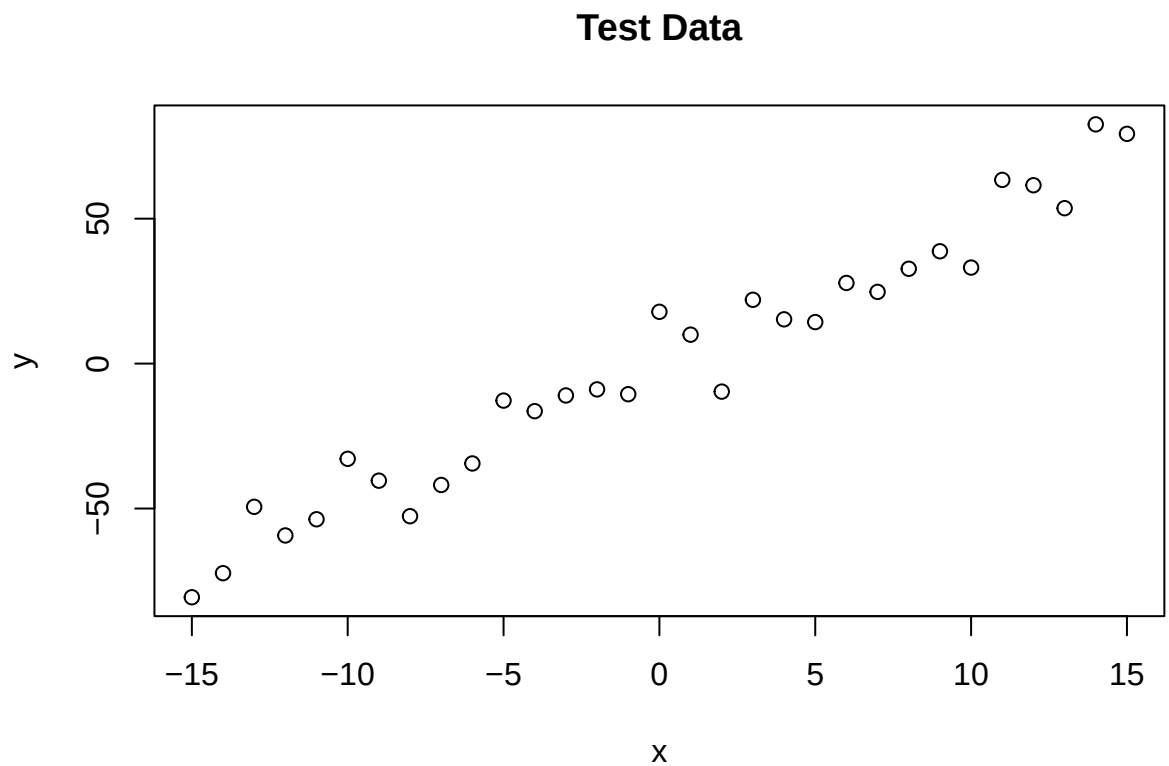
Problem 1

Consider the Metropolis-Hastings algorithm implemented in the attached file mcmc.R, where the parameters are a, b and sd and the goal is draw samples from the joint posterior distribution of the parameters. Submit all R code.

A.

[17 points] Consider the R function proposalfunction(). This currently employs a random walk Metropolis step with a Gaussian proposal. Change this function to implement an independent Metropolis-Hastings sampler where you draw the proposals from the respective prior distributions instead. You will need to modify the M-H acceptance ratio accordingly to have a correct sampler. Construct histograms for each parameters based on the posterior draws.

```r
# This work is licensed under the Creative Commons Attribution-NonCommercial-ShareAlike 3.0 Unported Li
# Example taken from http://theoreticalecology.wordpress.com/2010/09/17/metropolis-hastings-mcmc-in-r/
# True paramter values
trueA <- 5
trueB <- 0
trueSd <- 10
sampleSize <- 31

## Set the seed so that results can be replicated
set.seed(123)
# create x-values
x <- (-(sampleSize-1)/2):((sampleSize-1)/2)

# create dependent values according to y = ax + b + N(0,sd)
y <-trueA * x + trueB + rnorm(n=sampleSize,mean=0,sd=trueSd)
plot(x,y, main="Test Data")
```




```
####### Required fnctions

#likelihood function: Note that it returns "log" likelihood
likelihood <- function(param){
  a = param[1]
  b = param[2]
  sd = param[3]
  pred = a*x + b
  singlelikelihoods = dnorm(y, mean = pred, sd = sd, log = T)
  sumll = sum(singlelikelihoods)
  return(sumll)
}
# Example: plot the likelihood profile of the slope a
#slopevalues <- function(x){return(likelihood(c(x, trueB, trueSd)))}
#slopelikelihoods <- lapply(seq(3, 7, by=.05), slopevalues )
#plot (seq(3, 7, by=.05), slopelikelihoods , type="l", xlab = "values of slope parameter a", ylab = "Lo

# Prior likelihood function: Note that it returns "log" prior likelihood
prior <- function(param){
  a = param[1]
  b = param[2]
  sd = param[3]
  aprior = dunif(a, min=0, max=10, log = T)
  bprior = dnorm(b, sd = 5, log = T)
  sdprior = dunif(sd, min=0, max=30, log = T)
  return(aprior+bprior+sdprior)
```

```r
}

# The posterior
posterior <- function(param){
  return (likelihood(param) + prior(param))
}
```

We convert to independent samples metropolis.

We use proposal function that is just the priors, so we get:

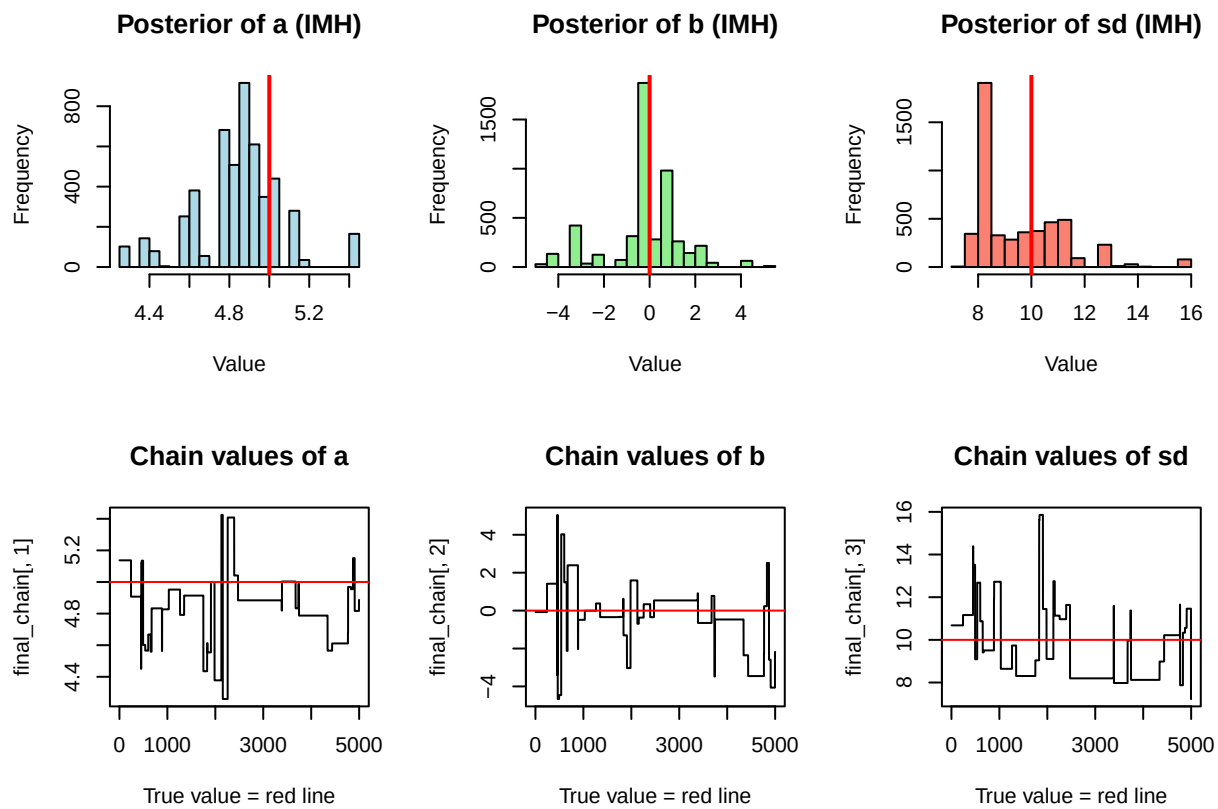
$$r = \min\left(1, \frac{P(y|\theta_{prop})P(\theta_{prop})}{P(y|\theta_{curr})P(\theta_{curr})} \times \frac{g(\theta_{curr})}{g(\theta_{prop})}\right)$$

But since we choose our proposal function to just be the priors, our proposal $g(\theta)$ is the prior $P(\theta)$, the priors and proposal densities cancel out each other. The acceptance ratio simplifies to the ratio of the posterior likelihoods by themselves.

Now we implement our MCMC method.

```r
set.seed(123)


# Function to draw Y_t ~ g(y)
proposalfunction_imh <- function(param){
  a_prop = runif(1, min=0, max=10)
  b_prop = rnorm(1, mean=0, sd=5)
  sd_prop = runif(1, min=0, max=30)
  return(c(a_prop, b_prop, sd_prop))
}

# Our MCMC step using previous function
run_independent_MH <- function(startvalue, iterations){
  chain = array(dim = c(iterations+1,3))
  chain[1,] = startvalue
  for (i in 1:iterations){
    proposal = proposalfunction_imh(chain[i,])

    probab = exp(likelihood(proposal) - likelihood(chain[i,]))

    if (runif(1) < probab){
      chain[i+1,] = proposal
    } else {
      chain[i+1,] = chain[i,]
    }
  }
  return(chain)
}

## Set starting values: a=4, b=0 and sd=10?
startvalue = c(4,0,10)
## Run the chain
chain_imh = run_independent_MH(startvalue, 10000)
```

```r
### Summary: #######################
burnIn = 5000
final_chain = chain_imh[-(1:burnIn),]
acceptance_imh = 1-mean(duplicated(final_chain))

par(mfrow = c(2,3))
hist(final_chain[,1], nclass=30, main="Posterior of a (IMH)", xlab="Value", col="lightblue")
abline(v = trueA, col="red", lwd=2)

hist(final_chain[,2], nclass=30, main="Posterior of b (IMH)", xlab="Value", col="lightgreen")
abline(v = trueB, col="red", lwd=2)

hist(final_chain[,3], nclass=30, main="Posterior of sd (IMH)", xlab="Value", col="salmon")
abline(v = trueSd, col="red", lwd=2)

plot(final_chain[,1], type = "l", xlab="True value = red line" , main = "Chain values of a", )
abline(h = trueA, col="red" )
plot(final_chain[,2], type = "l", xlab="True value = red line" , main = "Chain values of b", )
abline(h = trueB, col="red" )
plot(final_chain[,3], type = "l", xlab="True value = red line" , main = "Chain values of sd", )
abline(h = trueSd, col="red" )
```




```r
# for comparison:
summary(lm(y~x))
```

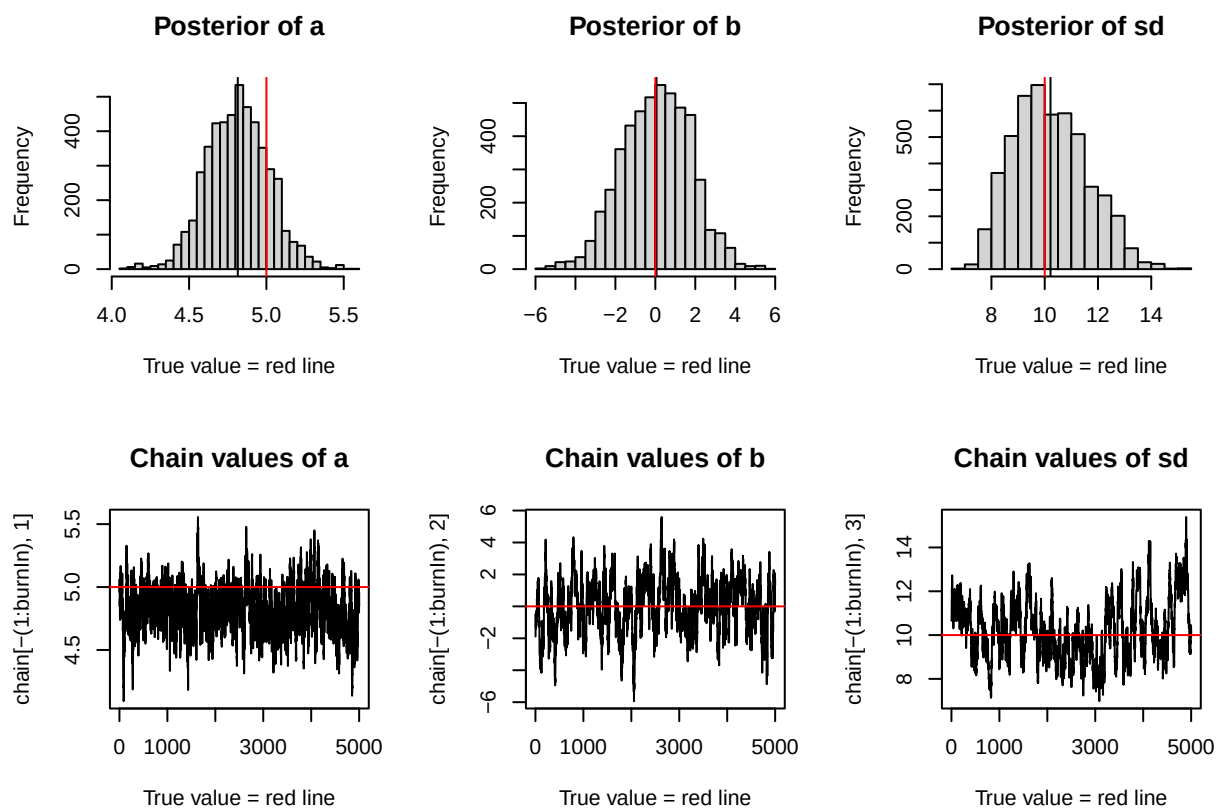
```
##
## Call:
## lm(formula = y ~ x)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -18.9593  -6.6725  -0.6956   6.4944  18.1874
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)  -0.3183     1.7391  -0.183    0.856
## x             4.8057     0.1944  24.715   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 9.683 on 29 degrees of freedom
## Multiple R-squared:  0.9547, Adjusted R-squared:  0.9531
## F-statistic: 610.9 on 1 and 29 DF,  p-value: < 2.2e-16
```

B.

[8 points] Plot the autocorrelation function of the posterior draws of the parameters in the new sampler. Compare with random walk Metropolis. Is there any appreciable difference in any of the ACFs?

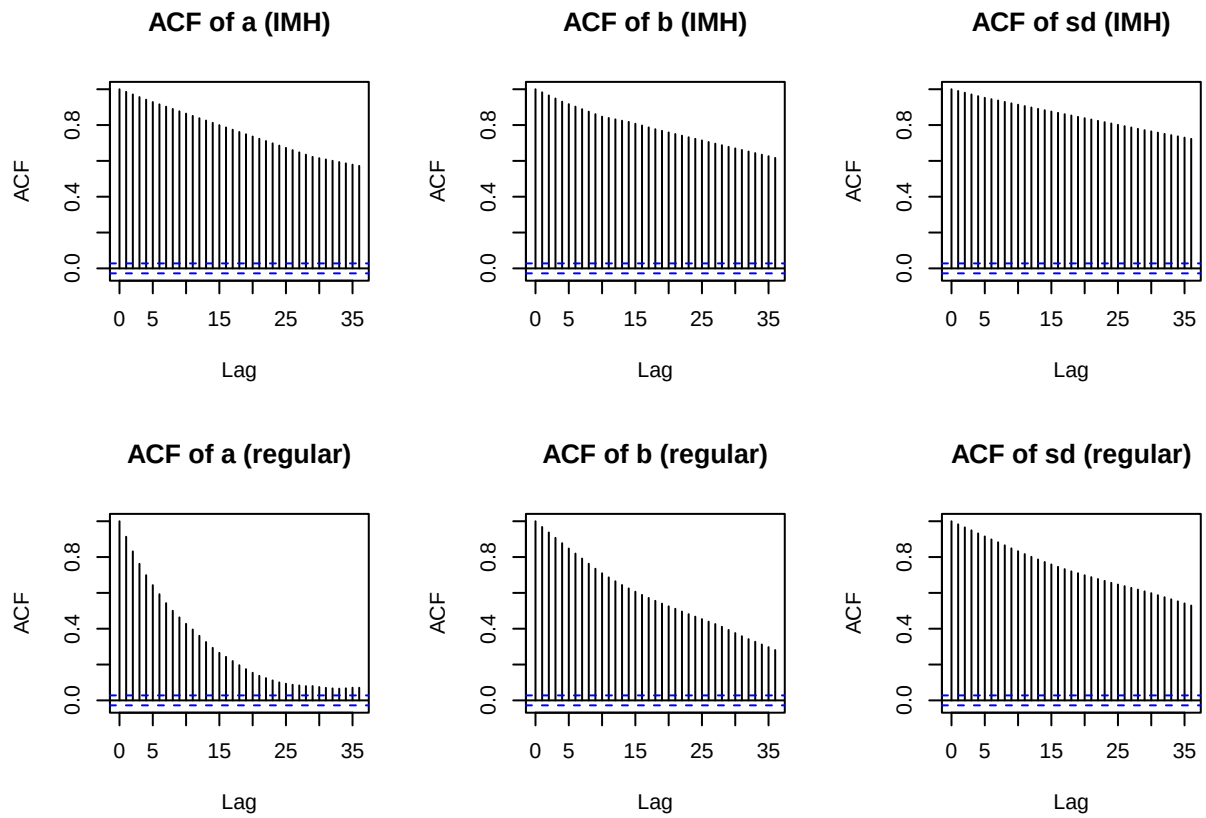
```r
######## Metropolis algorithm ################

proposalfunction <- function(param){
  return(rnorm(3,mean = param, sd= c(0.1,0.5,0.3)))
}

run_metropolis_MCMC <- function(startvalue, iterations){
  chain = array(dim = c(iterations+1,3))
  chain[1,] = startvalue
  for (i in 1:iterations){
    proposal = proposalfunction(chain[i,])
    probab = exp(posterior(proposal) - posterior(chain[i,]))
    if (runif(1) < probab){
      chain[i+1,] = proposal
    }else{
      chain[i+1,] = chain[i,]
    }
  }
  return(chain)
}

## Set starting values: a=4, b=0 and sd=10?
startvalue = c(4,0,10)

## Run the chain
chain = run_metropolis_MCMC(startvalue, 10000)

## Drop the burn-in samples?
```

```r
burnIn = 5000
acceptance = 1-mean(duplicated(chain[-(1:burnIn),]))

### Summary: #######################
par(mfrow = c(2,3))
hist(chain[-(1:burnIn),1],nclass=30, , main="Posterior of a", xlab="True value = red line" )
abline(v = mean(chain[-(1:burnIn),1]))
abline(v = trueA, col="red" )
hist(chain[-(1:burnIn),2],nclass=30, main="Posterior of b", xlab="True value = red line")
abline(v = mean(chain[-(1:burnIn),2]))
abline(v = trueB, col="red" )
hist(chain[-(1:burnIn),3],nclass=30, main="Posterior of sd", xlab="True value = red line")
abline(v = mean(chain[-(1:burnIn),3]) )
abline(v = trueSd, col="red" )
plot(chain[-(1:burnIn),1], type = "l", xlab="True value = red line" , main = "Chain values of a", )
abline(h = trueA, col="red" )
plot(chain[-(1:burnIn),2], type = "l", xlab="True value = red line" , main = "Chain values of b", )
abline(h = trueB, col="red" )
plot(chain[-(1:burnIn),3], type = "l", xlab="True value = red line" , main = "Chain values of sd", )
abline(h = trueSd, col="red" )
```




```r
par(mfrow = c(2,3))
acf(final_chain[,1], main="ACF of a (IMH)")
acf(final_chain[,2], main="ACF of b (IMH)")
acf(final_chain[,3], main="ACF of sd (IMH)")
```

```r
acf(chain[-(1:burnIn),1], main="ACF of a (regular)")
acf(chain[-(1:burnIn),2], main="ACF of b (regular)")
acf(chain[-(1:burnIn),3], main="ACF of sd (regular)")
```



Honestly, the autocorrelation is not good at all. The original algorithm performed much better in terms of autocorrelation. Perhaps the prior did not give a good random sample so we couldn't really properly traverse the state space. We see that we also accepted very few samples (0.0075) so this could likely be the case.

Problem 2

Consider the two stage Gibbs sampler for generating samples from the bivariate normal density

A.

[15 points] Consider the X chain. Write down the transition kernel $K(x \to x)$. Consequently, show that $X|X = x \sim Normal$

We go from x^* to y and then from y to x, so we can set up this transition kernel as the two steps:

$$K(x^* \to x) = \int p(x|y)p(y|x^*)\, dy$$

Again, we know $p(y|x^*) = \text{Normal}(\rho x^*, 1 - \rho^2)$ and $p(x|y) = \text{Normal}(\rho y, 1 - \rho^2)$, so now let us plug in and solve this integral.

$$K(x^* \to x) = \int_{-\infty}^{\infty} \frac{1}{\sqrt{2\pi(1-\rho^2)}} \exp\left(-\frac{(x-\rho y)^2}{2(1-\rho^2)}\right) \cdot \frac{1}{\sqrt{2\pi(1-\rho^2)}} \exp\left(-\frac{(y-\rho x^*)^2}{2(1-\rho^2)}\right) dy$$

Which kind of leads to a dead end because this is not really possible to integrate and the closed form does not seem to be normal. We will use alternative technique.

Again we use the same idea. We start with x^* and find $Y = \rho x^* + \epsilon_1$, where $\epsilon_1 \sim \text{Normal}(0, 1 - \rho^2)$.

Now we plug that Y in to get X: $X = \rho Y + \epsilon_2$, where $\epsilon_2 \sim \text{Normal}(0, 1 - \rho^2)$.

Combining and expanding the two steps we get a linear combination of normal distributions (which of course is linear):

$$X = \rho(\rho x^* + \epsilon_1) + \epsilon_2 = \rho^2 x^* + \rho\epsilon_1 + \epsilon_2$$

Given $E[\epsilon_1] = E[\epsilon_2] = 0$, we can then compute the mean:

$$E[X|X^* = x^*] = E[\rho^2 x^* + \rho\epsilon_1 + \epsilon_2] = \rho^2 x^*$$

Next, given that ϵ_1 and ϵ_2 are independent samples in the Gibbs steps, variance:

$$\text{Var}(X|X^* = x^*) = \text{Var}(\rho\epsilon_1) + \text{Var}(\epsilon_2) = \rho^2(1-\rho^2) + (1-\rho^2) = 1 - \rho^4$$

We know the linear combination of normal distributions is normal so our final distribution is

$$X|X^* = x^* \sim \text{Normal}(\rho^2 x^*, 1 - \rho^4)$$

B.

> [10 points] Assuming the starting value of the X chain is X0, derive an expression for $Cov(X_0, X_k)$. Deduce that the covariance goes to zero as k increases, as long as $|\rho| < 1$.

We need to derive the formula for covariance. We know previously the relationship between our current state, say X_k and its past state, X_{k-1} is simply $X_k = \rho^2 X_{k-1} + \epsilon_k$ where $\epsilon_k \sim \text{Normal}(0, 1-\rho^4)$ because $X_k | X_{k-1} \sim \text{Normal}(\rho^2 X_{k-1}, 1-\rho^4)$.

We can perform induction with this, starting at X_0 and going all the way to X_k. We can see a pattern:

Step 1: $X_1 = \rho^2 X_0 + \epsilon_1$

Step 2: $X_2 = \rho^2 X_1 + \epsilon_2 = \rho^2(\rho^2 X_0 + \epsilon_1) + \epsilon_2 = \rho^4 X_0 + \rho^2 \epsilon_1 + \epsilon_2$

Step 3: $X_3 = \rho^2 X_2 + \epsilon_3 = \rho^2(\rho^4 X_0 + \rho^2 \epsilon_1 + \epsilon_2) + \epsilon_3 = \rho^6 X_0 + \rho^4 \epsilon_1 + \rho^2 \epsilon_2 + \epsilon_3$

Step k: $X_k = \rho^{2k} X_0 + \sum_{i=1}^{k} \rho^{2(k-i)} \epsilon_i$

Now we can solve for covariance:

$$\begin{aligned} Cov(X_0, X_k) &= Cov\left(X_0, \rho^{2k} X_0 + \sum_{i=1}^{k} \rho^{2(k-i)} \epsilon_i\right) \\ &= Cov\left(X_0, \rho^{2k} X_0\right) + 0 \\ &= \rho^{2k} Var(X_0) \end{aligned}$$

If we started the chain with a sample from the bivariate normal density, then $Var(X_0) = 1$, but otherwise, we are not given the specific distributions of X_0.

Given that the variance of X_0 is finite, say some constant c, and $|\rho| < 1$, we can calculate the limit as k approaches infinity:

$$\lim_{k \to \infty} \rho^{2k} c = 0$$